Presentation
On
The Proposal
for

RepoLens

AI-Assisted Repository Intelligence for Security & Code Understanding

Group Number: 05

Presented by

Rounak Kumar Choudary • Rajat Negi • Priyanka Singh • Priyanshi Bora

Department of CSE
GNIOT (Greater Noida, India)

Under the Supervision
Mr. Imran Ansari

Outline

01 Introduction / Problem Statement

02 Existing Approaches / Related Work

03 Problems in Existing Approaches

04 Research Objectives

05 Proposed Methodology / Model

06 Expected Impact

07 Future Scope & Long-Term Vision

08 Current Limitations

09 Conclusion

Introduction / Problem Statement

The modern problem

Developers, contributors, and students often struggle to understand unfamiliar codebases. Existing tools frequently focus on vulnerability detection rather than explaining how risk is distributed across a repository.

Why current scanning is insufficient

Finding a vulnerability is only part of the task. Developers also need to know where risks are concentrated, why findings matter, how modules connect, and which files deserve review first.

Research motivation

Build repository intelligence that combines deterministic code analysis with AI-assisted reasoning, helping users understand what a repository does and what should be reviewed first.

Problem statement: “You open a repo and have no idea what is happening, where the risk is, or how modules connect.”

Existing Approaches / Related Work

Traditional code scanners

Primarily identify vulnerabilities and patterns. Useful for finding issues, but often provide limited repository-level context.

AI code assistants / scanners

Can explain code and findings, but sending large codebases directly to an LLM can increase token usage and make outputs harder to constrain and explain.

Repository analysis

AST-based analysis can extract structure, metadata, relationships, and call-graph information before higher-level reasoning is applied.

What the project builds on

- AST-based code analysis and pattern detection
- Security findings and metadata extraction
- Module relationships and call graphs
- AI-assisted repository-level reasoning

Problems in Existing Approaches

A repository scanner may tell you what is wrong — but not necessarily what matters first.

Finding overload

Large repositories can contain many findings. Without prioritization, developers may struggle to decide which files or modules deserve attention first.

Missing context

Individual findings do not fully explain repository structure, function relationships, or how a vulnerable component connects to other code.

LLM scalability

Sending large code files or entire repositories to an LLM increases token consumption and can make analysis less scalable and less controlled.

Research gap: move from isolated vulnerability detection toward structured, repository-level intelligence and risk prioritization.

Research Objectives

01

Understand

Extract repository structure, metadata, security findings, and relationships.

02

Analyze

Use deterministic AST-based analysis and pattern detection before AI reasoning.

03

Prioritize

Identify risk hotspots and connect findings into meaningful attack chains.

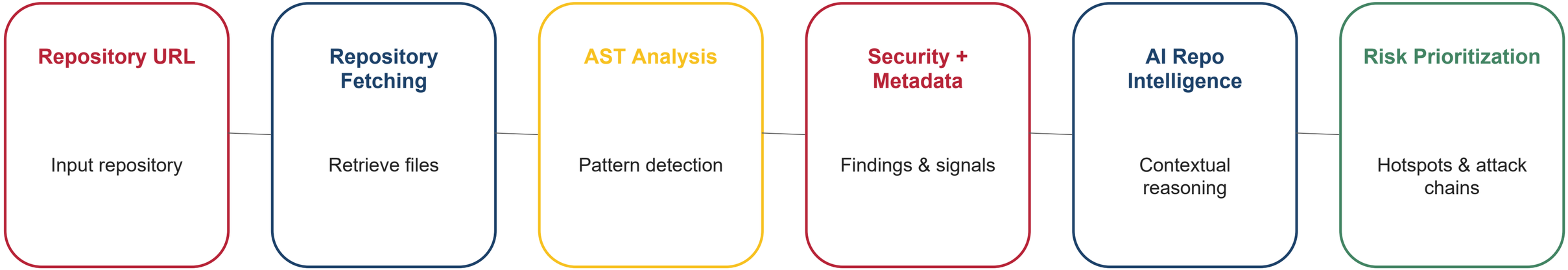
04

Explain

Use AI to turn structured repository intelligence into actionable understanding.

Core objective: transform raw code and scanner output into structured repository intelligence that supports security review and code understanding.

Proposed Methodology / Model



Why this architecture?

- Reduces token consumption
- Improves explainability
- Reduces hallucination risk
- Scales more effectively for larger repositories

Structured AI input

- Findings
- Metadata
- Module relationships
- Call graphs
- Risk signals

Repository → deterministic analysis → compact structured intelligence → AI reasoning → prioritized security/code understanding

Expected Impact & Current Limitations

For Contributors

- Faster onboarding into unfamiliar repositories
- Better understanding of repository architecture
- Clear identification of security-sensitive modules

For Development Teams

- Easier prioritization of code reviews
- Faster security assessments
- Improved security awareness

For Organizations

- Reduced time spent understanding inherited codebases
- Better visibility into repository structure and risk
- Improved developer productivity

Current limitations

- GitHub API rate and file-retrieval limits constrain repository analysis.
- LLM token/request limits constrain repository intelligence generation.
- Structured AST-derived summaries are used to keep AI input compact.

Conclusion and Future Scope

Advanced Repository Intelligence

- Cross-file dependency analysis
- Architecture visualization
- Security review recommendations
- Repository health scoring

Enterprise Platform

- Private repository support
- Team collaboration features
- Organization-wide repository intelligence
- Security governance dashboards

Agentic Security Workflows

- Automated repository review agents
- Continuous repository monitoring
- Pull request intelligence
- Security-aware contributor onboarding

Long-term vision: evolve from a repository scanner into an AI-powered repository intelligence platform that helps developers, teams, and organizations understand, secure, and maintain complex codebases at scale.